

ROS 2 Software Architecture — RoboLords 2026

1. Overview

The RoboLords 2026 vehicle runs on ROS 2 Humble. Rather than writing one large control program, we deliberately split the autonomous system into a small set of independent nodes, each owning a single responsibility. Sensing, decision-making and actuation therefore run as separate processes that communicate only through well-defined topics.

This decision shaped how the whole project was built. Because each node stands alone, team members were able to work on different parts of the robot in parallel, and any subsystem could be brought up, tested and tuned without the rest of the robot being present.

The autonomous stack is made up of five nodes:

Node	Responsibility
tf_luna_front	Measures clear distance ahead of the vehicle
tf_luna_rear_i2c	Measures clear distance behind the vehicle
imu_node	Supplies orientation and motion data
navigation_node	Interprets sensor data and decides how the robot should move
motor_node	Translates movement commands into drive and steering output

The camera system sits deliberately outside the ROS 2 graph. It supplies the colour information that drives our obstacle strategy, and keeping it separate meant vision work could progress on its own timeline without risk to the navigation stack.

2. System Architecture

Information moves through the system in one direction, from sensing to actuation:

```
flowchart LR
    A[TF-Luna Front] -->|/tf_luna/front| N[Navigation Node]
    B[TF-Luna Rear] -->|/tf_luna/rear| N
    C[BNO085 IMU] -->|/imu/data| N
    N -->|/cmd_vel| M[Motor Node]
    M --> D[Drive Motor]
    M --> E[Ackermann Steering]
```

Data flow

The pipeline follows a straightforward path: **sensors** → **navigation** → **motor control**.

The three sensor nodes publish continuously and independently. The navigation node consumes those streams, builds a picture of the robot's current situation, and decides what the vehicle should do next. That decision leaves the node as a single `geometry_msgs/Twist` message on `/cmd_vel`, which the motor node picks up and turns into physical motion.

One consequence of this design is worth highlighting: no sensor node knows anything about the motors, and the navigation node knows nothing about how motion is physically produced. Each layer only has to honour its contract with the next.

3. Sensor Nodes

3.1 TF-Luna Front Node

Node: `tf_luna_front`

This node reads the forward-facing TF-Luna and reports the distance between the front of the vehicle and whatever lies ahead of it — typically a wall or an obstacle.

Direction	Topic	Message type
Publishes	<code>/tf_luna/front</code>	<code>std_msgs/Float32</code>
Subscribes	—	—

Consumer: `navigation_node`

The forward reading is our primary cue for recognising that the robot is approaching a corner, and for detecting an obstacle early enough to plan around it rather than react to it.

3.2 TF-Luna Rear Node

Node: `tf_luna_rear_i2c`

This node reads the rear-facing TF-Luna over I²C and reports clearance behind the vehicle.

Direction	Topic	Message type
Publishes	<code>/tf_luna/rear</code>	<code>std_msgs/Float32</code>
Subscribes	—	—

Consumer: `navigation_node`

Rear distance matters most during manoeuvres where the robot has to reason about what is behind it. Parking is the clearest case, as the rear reading is what tells the robot how much room it still has as it backs into position.

4. IMU Node

Node: imu_node

Orientation and motion data come from an Adafruit BNO085 IMU.

Direction	Topic	Message type
Publishes	/imu/data	sensor_msgs/Imu
Subscribes	—	—

Consumer: navigation_node

The IMU is what allows the robot to turn by *angle* rather than by *time*. Early in development we timed our turns, and the results drifted with battery voltage, surface grip and load. Heading feedback removed that fragility.

For a 90° corner, the navigation node records the yaw at the start of the turn, watches the heading change as the vehicle rotates, and ends the manoeuvre once the target angle has actually been reached. The same principle is reused for obstacle avoidance turns.

5. Navigation Node

Node: navigation_node

The navigation node is the robot's decision-maker. It is the only node that sees the full sensor picture, and the only node that issues movement commands.

5.1 Inputs

Topic	Source	Purpose
/imu/data	imu_node	Orientation and heading
/tf_luna/front	tf_luna_front	Forward clearance
/tf_luna/rear	tf_luna_rear_i2c	Rear clearance

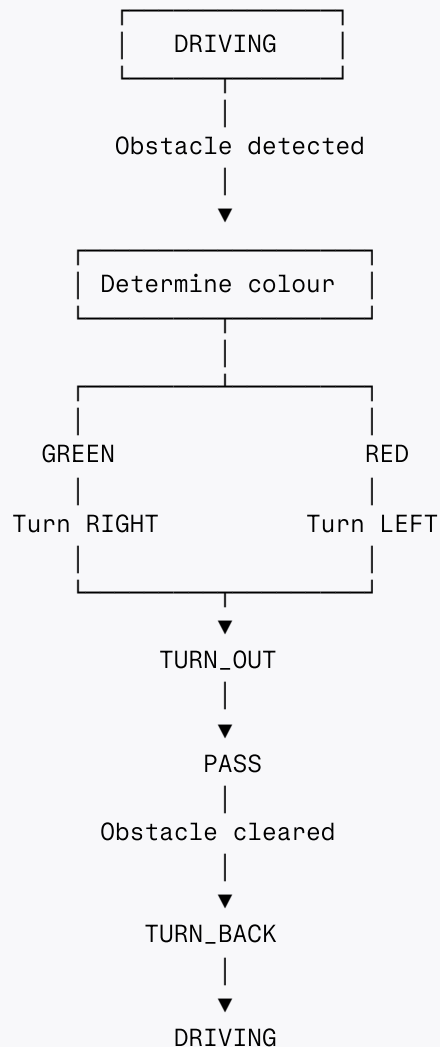
5.2 Output

Topic	Message type	Purpose
/cmd_vel	geometry_msgs/Twist	Linear and angular motion request

Everything the motor node needs is carried in that one Twist message. Keeping the interface this narrow meant we could change navigation behaviour repeatedly without touching hardware code.

6. Navigation Logic

Navigation is implemented as a **state machine**. Each state has a clear entry condition and a clear exit condition, which made behaviour predictable during testing and made faults easy to localise – when the robot misbehaved, we could usually name the state it was stuck in.



7. Obstacle Avoidance

Obstacle colour is identified by the camera system and drives the choice of direction, following the competition rule:

Colour	Pass on
☒ Green	Right
☒ Red	Left

Once an obstacle is detected, the navigation node runs a fixed avoidance sequence:

1. Detect the obstacle in the robot's path.
2. Determine its colour.

3. Select the required passing direction.
4. Rotate away from the obstacle.
5. Drive past it.
6. Confirm the obstacle has been cleared.
7. Rotate back towards the original heading.
8. Return to normal lap navigation.

Throughout steps 4 to 7 the IMU heading is the reference. The robot commits to a measured angular change rather than a guessed duration, which is what keeps the manoeuvre repeatable across runs.

8. Parking

Parking is handled by the navigation system using distance feedback and incremental movement commands, with the rear TF-Luna reporting how far the vehicle is from the rear boundary as the manoeuvre progresses.

```
graph TD
    A[Approach parking area] --> B[Begin parking manoeuvre]
    B --> C[Monitor rear distance]
    C --> D[Adjust movement]
    D --> E[Reach target position]
    E --> F[Stop]
```

We developed the parking routine separately from lap navigation on purpose. It has different tolerances and a different failure mode from open-course driving, so isolating it let us tune it aggressively without destabilising the lap behaviour we had already validated.

9. Motor Node

Node: `motor_node`

The motor node owns the drivetrain and the steering system, and nothing else.

Direction	Topic	Message type
Subscribes	<code>/cmd_vel</code>	<code>geometry_msgs/Twist</code>
Publishes	—	—

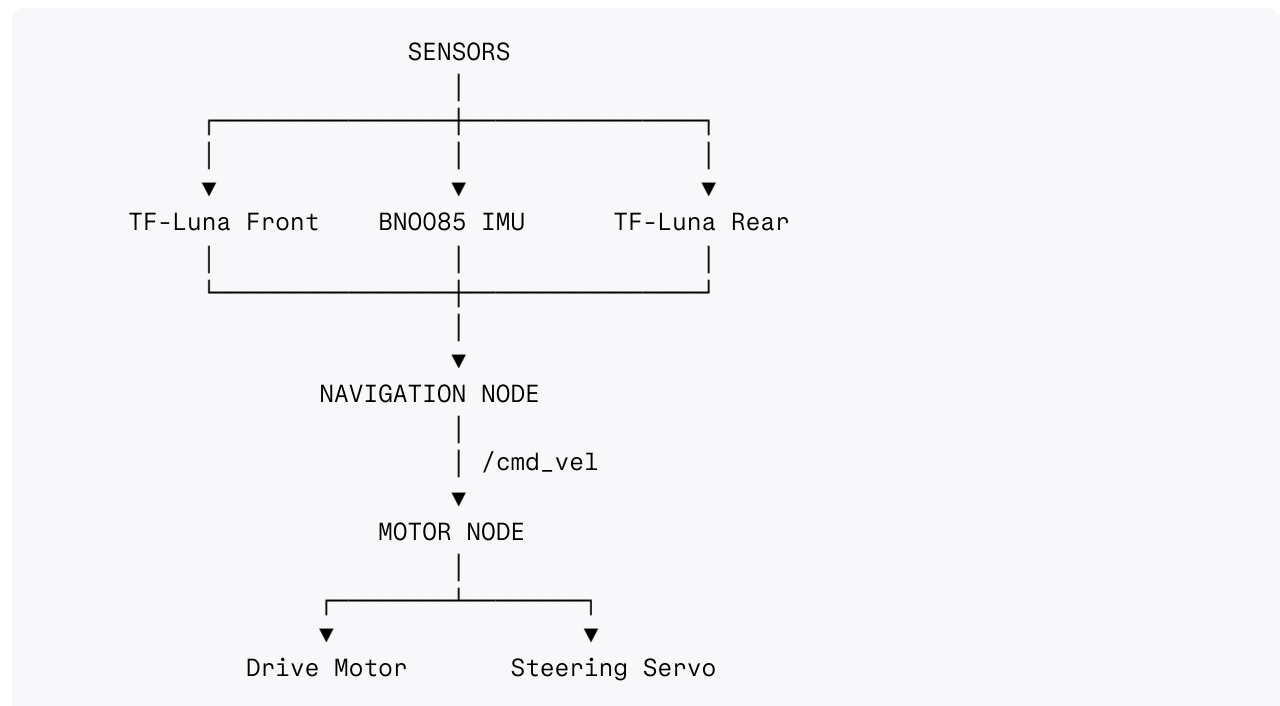
It receives motion requests and converts them into drive motor output and Ackermann steering output. Because this translation lives in one place, the navigation node never touches a GPIO pin or a PWM value — it states the motion it wants and the motor node works out how to produce it. Hardware changes stayed contained to this node.

10. Complete Topic Map

Publisher	Topic	Message type	Subscriber
tf_luna_front	/tf_luna/front	std_msgs/Float32	navigation_node
tf_luna_rear_i2c	/tf_luna/rear	std_msgs/Float32	navigation_node
imu_node	/imu/data	sensor_msgs/Imu	navigation_node
navigation_node	/cmd_vel	geometry_msgs/Twist	motor_node

The pattern is deliberate. The navigation node is the single junction between sensing and actuation, so there is exactly one place in the system where robot behaviour is decided.

11. Node Relationships



12. Why We Chose ROS 2

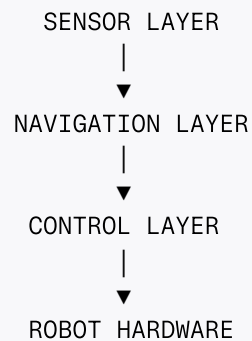
ROS 2 gave us a way to build the robot as a set of replaceable parts instead of one program that had to work all at once. In practice it delivered five things we relied on:

- **Modularity.** Every sensor and hardware subsystem has its own node with its own lifecycle, so a fault in one place does not force us to reason about the whole robot.
- **Independent testing.** We could launch a single node, echo its topic, and confirm it behaved correctly before it was ever integrated. Most of our bugs were caught this way rather than on the mat.
- **Standard communication.** Topics and standard message types gave us a common interface, so nodes written by different team members interoperated without negotiation.
- **Scalability.** Adding a sensor or a new behaviour means adding a publisher or a subscriber, not rewriting the control loop.

- **Separation of concerns.** Navigation reasons about intent; the motor node reasons about hardware. Neither has to understand the other.

Taken together, these let us develop the robot incrementally, which mattered a great deal under competition time pressure.

13. Architecture Summary



The final design is a clean three-layer hierarchy. The sensor layer reports what the robot can perceive, the navigation layer decides what it should do, and the control layer makes it happen.

That structure is the reason the RoboLords team could validate sensing, navigation and motor control as separate pieces of work and then bring them together into a functioning autonomous vehicle with confidence in each part.